GUIA TÉCNICO EP / DISCIPLINA 04

Cassandra: modelagem e CQL

Partições, clustering, tabelas por consulta e consistência

MAPA DE ESTUDO

01 Modelagem orientada à consulta

02 Chaves de partição e clustering

03 CQL e organização por consultas

04 Laboratório real de nó único

05 Limites de escala e replicação

Material de referência complementar aos notebooks. Exemplos locais com dados sintéticos; laboratórios externos somente sob configuração explícita.

10 | Cassandra: modelagem e CQL

GUIA TÉCNICO APROFUNDADO • ANÁLISE, CÓDIGO E INTERPRETAÇÃO

Objetivo e escopo

Estudo independente baseado no notebook aprovado. Os exemplos locais usam SQLite e dados sintéticos; PostgreSQL, MongoDB e Cassandra requerem serviços de laboratório para verificação real.

Conceito central: Chaves orientadas à consulta

Cassandra modela tabelas a partir das consultas: a chave de partição define co-localização e clustering columns ordenam linhas dentro da partição. Não projete CQL como se fosse SQL relacional. Evite ALLOW FILTERING como solução padrão.

Aplicação, ressalvas e limites

O esquema SimpleStrategy e fator de replicação 1 deste pacote são exclusivamente para laboratório de um nó. Não extrapole a configuração para produção.

10 - Cassandra: Modelagem e CQL

Objetivos: modelar tabelas orientadas às consultas e usar Cassandra real como prática opcional

Como estudar: leia a explicação, execute os exemplos, interprete os resultados, refaça os exercícios e consulte o resumo. Exemplos sintéticos são identificados; serviços externos são opcionais e precisam estar disponíveis para laboratórios de infraestrutura.

Cassandra e padrões de acesso

Cassandra é um banco wide-column distribuído: modelamos a partir das perguntas. A chave de partição define agrupamento; clustering columns ordenam os dados dentro da partição. Joins e consultas ad hoc não seguem o mesmo desenho do SQL relacional.

Exemplo do notebook — execução local

```
print((Path('sql')/'04_cassandra.cql').read_text(encoding='utf8'))
```

Estudo de caso - transporte

Exercício inspirado no problema de rotas e embarque/desembarque. C1: listar eventos por linha e dia, do mais recente. C2: listar eventos por ponto e dia. Criamos duas tabelas orientadas às consultas, duplicando dados intencionalmente.

Exemplo do notebook — execução local

```
c1='SELECT * FROM transporte.embarques_por_linha_dia WHERE linha=? AND dia=?
    '; c2='SELECT * FROM transporte.embarques_por_ponto_dia WHERE ponto=? AN
    D dia=?';print('C1:',c1,'\nC2:',c2)
```

Laboratório real opcional

Use docker compose -f infra/compose.yaml --profile cassandra up -d, aguarde o healthcheck, execute o script CQL via cqlsh. A consulta Python a seguir é habilitada apenas mediante confirmação explícita.

Exemplo do notebook — laboratório opcional, exige serviço isolado

```
# A execução local não ativa conexões automaticamente.
# Para laboratórios opcionais, configure RUN_LIVE_DB=1 antes de iniciar o ke
    rnel,
# depois de verificar que a instância é EXCLUSIVAMENTE de estudos.
import os
print('Laboratório externo:', 'habilitado' if os.getenv('RUN_LIVE_DB') == '1
    ' else 'desabilitado (padrão seguro)')
```

Exemplo do notebook — laboratório opcional, exige serviço isolado

```
import os
if os.getenv('RUN_LIVE_DB')=='1':
 from cassandra.cluster import Cluster
 from cassandra.query import SimpleStatement
 cluster=Cluster(['127.0.0.1'],port=9042,connect_timeout=8)
 session=cluster.connect()
 print('Cluster:',session.execute('SELECT release_version FROM system.local'
    ).one())
 session.execute("CREATE KEYSPACE IF NOT EXISTS transporte WITH replication
    = {'class':'SimpleStrategy','replication_factor':1}")
 cluster.shutdown()
else:
 print('Cassandra não executado: é necessário contêiner real e RUN_LIVE_DB=1
    .')
```

Exercício resolvido - particionamento

Problema: uma consulta por linha e dia deve evitar varrer toda a base. Solução: (linha,dia) é a partition key, horario a clustering key. Limitação: linhas com volume extremamente desigual exigem avaliar partições grandes e buckets.

Exemplo do notebook — execução local

```
print('Chave de partição: (linha, dia); ordenação dentro da partição: horari
    o DESC')
```

Pontos de atenção

SimpleStrategy e fator de replicação 1 são exclusivos do laboratório local, NÃO recomendações para produção. Sem múltiplos nós, não afirmaremos observar tolerância a falhas distribuída.

--- ### Aprofundamento v2 - Modelagem orientada à consulta

No Cassandra, a chave de partição define o agrupamento lógico; as *clustering columns* ordenam linhas dentro dessa partição. Uma chave popular demais cria partições quentes ou excessivamente grandes. Dados podem ser duplicados deliberadamente em diferentes tabelas para responder a consultas distintas.

Interpretação: compare o resultado com a teoria anterior; este exemplo local não substitui um laboratório de serviço externo.

Exemplo do notebook — execução local

```
eventos=[('L1','2026-11-09',8,'P1'),('L1','2026-11-09',9,'P2'),('L2','2026-1
    1-09',10,'P1')]
por_linha={}
for linha,dia,hora,ponto in eventos:
```

```
    por_linha.setdefault((linha,dia),[]).append((hora,ponto))
assert len(por_linha[('L1','2026-11-09')])==2
print('Modelo lógico por chave (linha,dia):',por_linha)
```

Exercício de revisão

Exercício resolvido - revisão: identifique a principal hipótese do exemplo acima, explique por que ela importa e cite uma condição na qual a técnica falharia. Resposta orientadora: registre a pré-condição (tipo de dado, chave, restrição, volume ou topologia), teste-a explicitamente e explique os limites dos resultados observados.

Aprofundamento: decisões e estudo de caso

Partição e clustering

Uma tabela Cassandra precisa ser projetada segundo padrões de leitura concretos. A partition key determina quais linhas ficam agrupadas logicamente; clustering columns organizam linhas dentro da partição. Escolher apenas a chave com maior cardinalidade não basta: partições muito largas, distribuição desigual e consultas sem chave podem comprometer eficiência.

Uma tabela por pergunta

É comum manter tabelas distintas para “embarques por linha e dia” e “embarques por ponto e dia”, pois Cassandra não oferece a mesma flexibilidade de joins de um banco relacional. A duplicação controlada serve às leituras prioritárias, mas precisa de política de escrita consistente e conhecimento sobre atualização de dados.

CQL e ambiente

WHERE deve respeitar a estrutura das chaves; filtros arbitrários podem não ser suportados ou exigir varreduras custosas. A configuração SimpleStrategy e fator de replicação 1 é adequada apenas para praticar localmente com um nó. Em produção, fatores de replicação e topologia devem refletir disponibilidade, tolerância a falhas e data centers.

Exercício orientado à consulta

Esboce duas tabelas para recuperar viagens por linha e por ponto, sempre dentro de um dia. Identifique partition key e clustering column, ordem desejada e possível tamanho da partição. Depois descreva quais consultas não seriam atendidas sem outra tabela ou estratégia de índice, sem usar ALLOW FILTERING como solução automática.

Checklist de interpretação

- Defina a pergunta, unidade de análise e pré-condições da consulta.
- Registre chaves, cardinalidade, restrições e tratamento de valores ausentes.
- Execute os exemplos locais antes de habilitar serviços reais.
- Separe evidência observada de hipóteses e propriedades não testadas.

Leituras e arquivos de apoio

Consulte o notebook correspondente, os scripts em sql/, os datasets sintéticos e a documentação oficial dos SGBDs. Os resultados locais não substituem testes de integração com serviços externos.